

ADR-006-database-schema-design

ADR-006: Database Schema Design

Status

Accepted

Date

2026-01-08

Context

HelixCode requires persistent storage for:

1. **User Management:** Users, authentication, sessions
2. **Distributed Computing:** Workers, metrics, connectivity events
3. **Task Management:** Distributed tasks, checkpoints, dependencies
4. **Project Management:** Projects, sessions, workflows
5. **LLM Integration:** Providers, models, configurations
6. **MCP Integration:** Servers, tools, permissions
7. **Notifications:** Multi-channel notification tracking
8. **Audit Logging:** Security and operational auditing

The database must support: - High concurrency for real-time operations - Complex queries for analytics - JSON storage for flexible schemas - UUID primary keys for distributed generation - Proper indexing for performance - Referential integrity for data consistency

Decision

We chose PostgreSQL as the primary database with a comprehensive relational schema, utilizing PostgreSQL-specific features like JSONB, arrays, and UUIDs.

Database Technology Choice

PostgreSQL 14+ with extensions: - `uuid-oss`: UUID generation - `pgcrypto`: Cryptographic functions

Schema Organization

The schema is organized into 8 logical domains:

1. Users & Authentication

```
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

username VARCHAR(255) UNIQUE NOT NULL,
email VARCHAR(255) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
avatar_url TEXT,
is_active BOOLEAN NOT NULL DEFAULT true,
is_verified BOOLEAN NOT NULL DEFAULT false,
mfa_enabled BOOLEAN NOT NULL DEFAULT false,
mfa_secret VARCHAR(255),
last_login TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE user_sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    session_token VARCHAR(512) UNIQUE NOT NULL,
    client_type VARCHAR(50) NOT NULL
        CHECK (client_type IN ('terminal_ui', 'cli', 'rest_api',
            'mobile_ios', 'mobile_android')),
    ip_address INET,
    user_agent TEXT,
    expires_at TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

2. Workers & Distributed Computing

```

CREATE TABLE workers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    hostname VARCHAR(255) NOT NULL,
    display_name VARCHAR(255),
    ssh_config JSONB NOT NULL,
    capabilities TEXT[] NOT NULL DEFAULT '{}',
    resources JSONB NOT NULL DEFAULT '{}',
    status VARCHAR(50) NOT NULL DEFAULT 'active'
        CHECK (status IN ('active', 'inactive', 'maintenance',
            'failed', 'offline')),
    health_status VARCHAR(50) NOT NULL DEFAULT 'healthy'
        CHECK (health_status IN ('healthy', 'degraded',
            'unhealthy', 'unknown')),
    last_heartbeat TIMESTAMPTZ,
    cpu_usage_percent DECIMAL(5,2),
    memory_usage_percent DECIMAL(5,2),
    disk_usage_percent DECIMAL(5,2),
    current_tasks_count INTEGER NOT NULL DEFAULT 0,
    max_concurrent_tasks INTEGER NOT NULL DEFAULT 10,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```

        updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

CREATE TABLE worker_metrics (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    worker_id UUID NOT NULL REFERENCES workers(id) ON DELETE
        CASCADE,
    cpu_usage_percent DECIMAL(5,2),
    memory_usage_percent DECIMAL(5,2),
    disk_usage_percent DECIMAL(5,2),
    network_rx_bytes BIGINT,
    network_tx_bytes BIGINT,
    current_tasks_count INTEGER,
    temperature_celsius DECIMAL(5,2),
    recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

3. Distributed Tasks

```

CREATE TABLE distributed_tasks (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    task_type VARCHAR(100) NOT NULL,
    task_data JSONB NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'assigned', 'running',
            'completed', 'failed', 'paused', 'waiting_for_worker')),
    priority INTEGER NOT NULL DEFAULT 5,
    criticality VARCHAR(20) NOT NULL DEFAULT 'normal'
        CHECK (criticality IN ('low', 'normal', 'high',
            'critical')),
    assigned_worker_id UUID REFERENCES workers(id),
    original_worker_id UUID REFERENCES workers(id),
    dependencies UUID[] DEFAULT '{}',
    retry_count INTEGER NOT NULL DEFAULT 0,
    max_retries INTEGER NOT NULL DEFAULT 3,
    error_message TEXT,
    result_data JSONB,
    checkpoint_data JSONB,
    estimated_duration INTERVAL,
    started_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE task_checkpoints (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```

    task_id UUID NOT NULL REFERENCES distributed_tasks(id) ON
        DELETE CASCADE,
    checkpoint_name VARCHAR(255) NOT NULL,
    checkpoint_data JSONB NOT NULL,
    worker_id UUID NOT NULL REFERENCES workers(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

4. Projects & Sessions

```

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    owner_id UUID NOT NULL REFERENCES users(id),
    workspace_path TEXT,
    git_repository_url TEXT,
    config JSONB NOT NULL DEFAULT '{}',
    status VARCHAR(50) NOT NULL DEFAULT 'active'
        CHECK (status IN ('active', 'archived', 'deleted')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    session_type VARCHAR(50) NOT NULL
        CHECK (session_type IN ('planning', 'building',
            'testing', 'refactoring', 'debugging')),
    status VARCHAR(50) NOT NULL DEFAULT 'active'
        CHECK (status IN ('active', 'paused', 'completed',
            'failed', 'waiting_for_worker')),
    context_data JSONB NOT NULL DEFAULT '{}',
    token_count INTEGER NOT NULL DEFAULT 0,
    current_task_id UUID REFERENCES distributed_tasks(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

5. LLM Providers & Models

```

CREATE TABLE llm_providers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) UNIQUE NOT NULL,
    provider_type VARCHAR(50) NOT NULL

```

```

        CHECK (provider_type IN ('local', 'openai', 'anthropic',
                                'gemini', 'qwen', 'xai', 'openrouter', 'copilot',
                                'custom')),
    api_key TEXT,
    api_endpoint TEXT,
    config JSONB NOT NULL DEFAULT '{}',
    status VARCHAR(50) NOT NULL DEFAULT 'active'
        CHECK (status IN ('active', 'inactive', 'failed')),
    health_status VARCHAR(50) NOT NULL DEFAULT 'unknown'
        CHECK (health_status IN ('healthy', 'degraded',
                                'unhealthy', 'unknown')),
    last_health_check TIMESTAMPTZ,
    error_count INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE llm_models (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    provider_id UUID NOT NULL REFERENCES llm_providers(id) ON
        DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    model_id VARCHAR(255) NOT NULL,
    capabilities TEXT[] NOT NULL DEFAULT '{}',
    context_length INTEGER NOT NULL DEFAULT 4096,
    max_tokens INTEGER NOT NULL DEFAULT 2048,
    supports_tools BOOLEAN NOT NULL DEFAULT false,
    supports_vision BOOLEAN NOT NULL DEFAULT false,
    description TEXT,
    status VARCHAR(50) NOT NULL DEFAULT 'active'
        CHECK (status IN ('active', 'inactive', 'deprecated')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

6. MCP Integration

```

CREATE TABLE mcp_servers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) UNIQUE NOT NULL,
    transport_type VARCHAR(50) NOT NULL
        CHECK (transport_type IN ('stdio', 'sse', 'http',
                                'websocket')),
    command TEXT,
    args TEXT[],
    url TEXT,
    env_vars JSONB NOT NULL DEFAULT '{}',
    status VARCHAR(50) NOT NULL DEFAULT 'active'
);

```

```

        CHECK (status IN ('active', 'inactive', 'failed')),
        last_health_check TIMESTAMPTZ,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

```

```

CREATE TABLE tools (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    mcp_server_id UUID REFERENCES mcp_servers(id) ON DELETE
        CASCADE,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    parameters JSONB NOT NULL DEFAULT '{}',
    permissions TEXT[] NOT NULL DEFAULT '{}',
    is_enabled BOOLEAN NOT NULL DEFAULT true,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

7. Notifications

```

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    notification_type VARCHAR(50) NOT NULL
        CHECK (notification_type IN ('info', 'warning', 'error',
            'success', 'alert')),
    priority VARCHAR(50) NOT NULL DEFAULT 'medium'
        CHECK (priority IN ('low', 'medium', 'high', 'urgent')),
    channels TEXT[] NOT NULL DEFAULT '{}',
    status VARCHAR(50) NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'sent', 'failed',
            'cancelled')),
    metadata JSONB NOT NULL DEFAULT '{}',
    sent_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

8. Audit Logs

```

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(255) NOT NULL,
    resource_type VARCHAR(100) NOT NULL,
    resource_id UUID,

```

```

details JSONB NOT NULL DEFAULT '{}',
ip_address INET,
user_agent TEXT,
status VARCHAR(50) NOT NULL
    CHECK (status IN ('success', 'failure', 'error')),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

Indexing Strategy

Primary patterns: - B-tree indexes on foreign keys - B-tree indexes on status/type columns - GIN indexes on JSONB columns - GIN indexes on array columns - Composite indexes for common query patterns

```

-- Example indexes
CREATE INDEX workers_status_idx ON workers (status);
CREATE INDEX workers_capabilities_idx ON workers USING GIN
    (capabilities);
CREATE INDEX distributed_tasks_dependencies_idx ON
    distributed_tasks USING GIN (dependencies);
CREATE INDEX audit_logs_created_at_idx ON audit_logs
    (created_at);

```

Schema Auto-Initialization

Schema is automatically created on startup if it doesn't exist:

```

func (db *Database) InitializeSchema() error {
    // Check if schema exists
    var schemaExists bool
    err := db.Pool.QueryRow(ctx, `
        SELECT EXISTS(
            SELECT 1 FROM information_schema.tables
            WHERE table_schema = 'public' AND table_name =
            'users'
        )
    `).Scan(&schemaExists)

    if !schemaExists {
        _, err = db.Pool.Exec(ctx, createSchemaSQL)
    }
    return err
}

```

Connection Pool Configuration

```

poolConfig.MaxConns = 20
poolConfig.MinConns = 5

```

```
poolConfig.MaxConnLifetime = time.Hour  
poolConfig.MaxConnIdleTime = 30 * time.Minute
```

Consequences

Positive

1. **Data Integrity:** CHECK constraints enforce valid states
2. **Referential Integrity:** Foreign keys maintain relationships
3. **Flexibility:** JSONB for evolving schemas
4. **Performance:** Strategic indexing for query patterns
5. **Scalability:** PostgreSQL handles high concurrency
6. **UUID Keys:** No coordination needed for ID generation
7. **Time Zones:** TIMESTAMPTZ for proper timezone handling
8. **Arrays:** Native array support for capabilities/tags

Negative

1. **Complexity:** Many tables and relationships to manage
2. **Migration Burden:** Schema changes require migrations
3. **PostgreSQL Lock-in:** Uses PostgreSQL-specific features
4. **Index Maintenance:** Many indexes add write overhead
5. **JSONB Queries:** Less efficient than normalized columns

Neutral

1. **Learning Curve:** Team needs PostgreSQL expertise
2. **Operational Burden:** Requires PostgreSQL administration

Alternatives Considered

Alternative 1: Document Database (MongoDB)

Description: Use MongoDB for all storage.

Pros: - Flexible schema - Native JSON support - Horizontal scaling - Developer-friendly

Cons: - No referential integrity - Complex transactions - Query limitations - Different operational model

Why Rejected: Relational data (users, workers, tasks) benefits from referential integrity. JSONB provides flexibility where needed.

Alternative 2: Multi-Database Architecture

Description: Use PostgreSQL for relational data, Redis for sessions, MongoDB for documents.

Pros: - Optimized per use case - Better performance for specific patterns - Technology diversity

Cons: - Operational complexity - Consistency challenges - Multiple systems to manage - Higher infrastructure cost

Why Rejected: PostgreSQL with JSONB handles all use cases adequately. Simplicity preferred for initial architecture.

Alternative 3: Event Sourcing with PostgreSQL

Description: Store all changes as events, derive current state.

Pros: - Complete audit trail - Time travel queries - Event-driven architecture - Natural CQRS fit

Cons: - Increased storage - Complex queries for current state - Learning curve - Eventual consistency

Why Rejected: Traditional CRUD with audit logs provides sufficient auditability with simpler implementation.

Alternative 4: SQLite for Single-Node

Description: Use SQLite for simpler deployments.

Pros: - Zero configuration - No server needed - File-based storage - Embedded database

Cons: - Limited concurrency - No horizontal scaling - Feature limitations - Not production-grade for multi-user

Why Rejected: Enterprise requirements need PostgreSQL capabilities. Configuration difference minimal with containerization.

Implementation Notes

- Schema defined in `internal/database/database.go`
- Uses `pgx/v5` driver for PostgreSQL
- Connection pool automatically managed
- Schema auto-initialization on startup
- Column migrations handled incrementally

Migration Strategy

For schema evolution: 1. Add new columns with defaults 2. Create new tables without breaking dependencies 3. Migrate data in application code 4. Remove deprecated columns in future release

Related Decisions

- ADR-002: Distributed Worker Architecture (worker tables)
- ADR-005: Authentication System (user/session tables)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/database/database.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/database/interface.go`